Task 1: Migrating a Local MySQL Database to Azure Database for MySQL with Minimal Downtime

Source Database and Target Environment

For this task I used a local MySQL 8.0 instance running on Ubuntu, with an application database called appdb. It's around forty gigabytes, mostly transactional tables plus a few bigger reporting ones. I picked Azure Database for MySQL Flexible Server as the target rather than just spinning up a VM and installing MySQL on it myself, mainly because I didn't want to also be responsible for patching and backups on top of everything else once it's actually running.

Backup Method

I started with Percona XtraBackup for the initial backup, since it can back up InnoDB tables without locking them, which matters if the database can't just be taken offline.

```
xtrabackup --backup --target-dir=/backup/full --datadir=/var/lib/mysql
xtrabackup --prepare --target-dir=/backup/full
```

This is where I hit my first real problem though. I assumed I could just take this backup and load it straight into Azure, but Flexible Server won't accept a physical backup like that, it only takes logical imports. So the actual data that ended up in Azure came from a mysqldump export instead, taken with the master data flag so it records the binary log position at the exact moment of the dump, which I needed later for replication.

```
mysqldump --single-transaction --routines --triggers --master-data=2 -u
admin -p appdb > appdb_dump.sql
```

Looking back, XtraBackup ended up mostly useful as a fast local backup I could roll back to if something went wrong, rather than the actual migration tool, which isn't quite what I expected going in. I went with open source tools here rather than whatever migration wizard Azure offers, partly because that's what the brief asked for, but also because Dunbar (2024) makes a point about how relying on one provider's own migration tooling can end up locking you into that provider without really noticing it happening. Using mysqldump and XtraBackup at least keeps the process itself something I could point at a different cloud provider if I had to.

Data Transfer and Restoration

I uploaded the dump to Azure Blob Storage with azcopy and then imported it directly into the Flexible Server instance.

```
azcopy copy "appdb_dump.sql"
"https://plremigration.blob.core.windows.net/migration/appdb_dump.sql"
mysql -h plre-flexserver.mysql.database.azure.com -u admin -p appdb <
appdb_dump.sql
```

Reducing Downtime Through Replication

For the downtime side of things, I used Flexible Server's data in replication feature, which lets it act as a replica of the local server using normal MySQL binary log replication, once the dump has been loaded and you know the binlog position it was taken at.

```
CALL mysql.az_replication_change_master('source.plre.local', 'repl_user',
'password', 3306, 'mysql-bin.000045', 154, 0);
CALL mysql.az_replication_start;
```

This meant changes made on the local database after the dump kept streaming across instead of everything relying on one long blocking transfer. The actual downtime window ended up being quite short, just stopping writes briefly, checking replication lag had caught up to zero, and then switching the application's connection string over to Azure.

Validation

Once the cutover was done I compared row counts on the bigger tables and ran pt table checksum to check the data actually matched between the old and new database, then did a manual check of the application itself against the new endpoint before I was happy calling it done.

Challenges

The biggest issue for me was realising partway through that XtraBackup wasn't going to work the way I'd planned, since I hadn't fully appreciated that a managed service like Flexible Server only takes logical imports until I actually tried it. Getting the binlog position right in the mysqldump command also took more care than I expected, since if that number is wrong you end up with a gap between the dump and where replication actually starts, and nothing you check afterwards would necessarily catch that until something didn't line up.

Task 2: Security Risk Assessment for a Hypothetical Cloud Deployment

Scenario

A fintech startup is moving customer account and transaction data from an on premises system to a multi tenant public cloud database service, with customers based across the UK and EU.

Key Risks

The first risk I'd flag is data residency. Under GDPR, moving customer data across borders needs to be properly justified, and I think this is easy to miss with a managed database service, since the provider might replicate data across regions for availability reasons without that lining up with where you're actually allowed to have the data sitting.

The second is access control being misconfigured. In a shared responsibility model the provider looks after the underlying platform, but things like access policies and database credentials are still down to the customer, and a database left publicly reachable with weak authentication is a pretty common way breaches like this actually happen (OWASP, 2021).

I'd also flag vendor lock-in as a compliance issue, not just a technical one. If a startup builds its whole pipeline around one provider's own tools, it might struggle to show a regulator or a bigger customer that it actually has a workable exit plan if it needed one. Dunbar (2024) makes a similar point about DevOps tooling specifically, that this kind of lock-in risk often isn't obvious at the point the decision gets made.

Compliance Challenges

Beyond GDPR, if any card data is involved there's also PCI DSS to think about, and I think that gets harder to demonstrate in a multi tenant environment where you can't really audit the physical infrastructure yourself. ISO/IEC 27001 gives a structure for organising controls like this, but I don't think certification on its own actually resolves the tension between using a cloud platform that changes quickly and needing a compliance position that stays stable and auditable.

Mitigation

Encrypting data at rest and in transit, applying least privilege access properly, and being explicit about how data residency is configured against GDPR would deal

with the most obvious risks here. Longer term, I think keeping a documented exit plan and not building too deeply into provider specific features would help with the lock-in risk Dunbar (2024) raises, so the compliance position doesn't fall apart if the platform itself ever needs to change.

References

Dunbar, W. (2024) *Vendor-Locked DevOps Strategies*. New York: Association for Computing Machinery.

OWASP (2021) *OWASP Top 10:2021*. Available at: https://owasp.org/Top10/2021/ (Accessed: 27 September 2026).